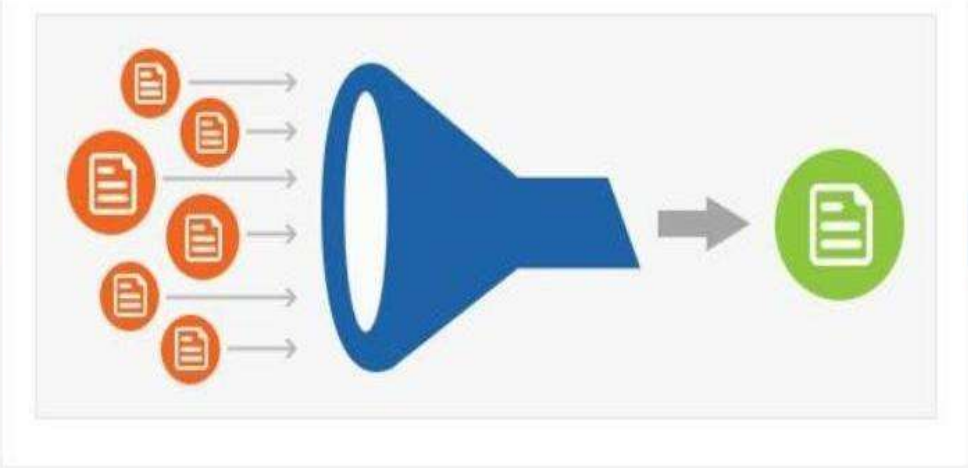
Data Pre-processing Techniques – 1

Introduction to Data Pre-processing Techniques

What is Data Pre-processing?

Raw data is often incomplete, inconsistent, and may also contain errors. Data pre-processing is a technique that resolves these issues.

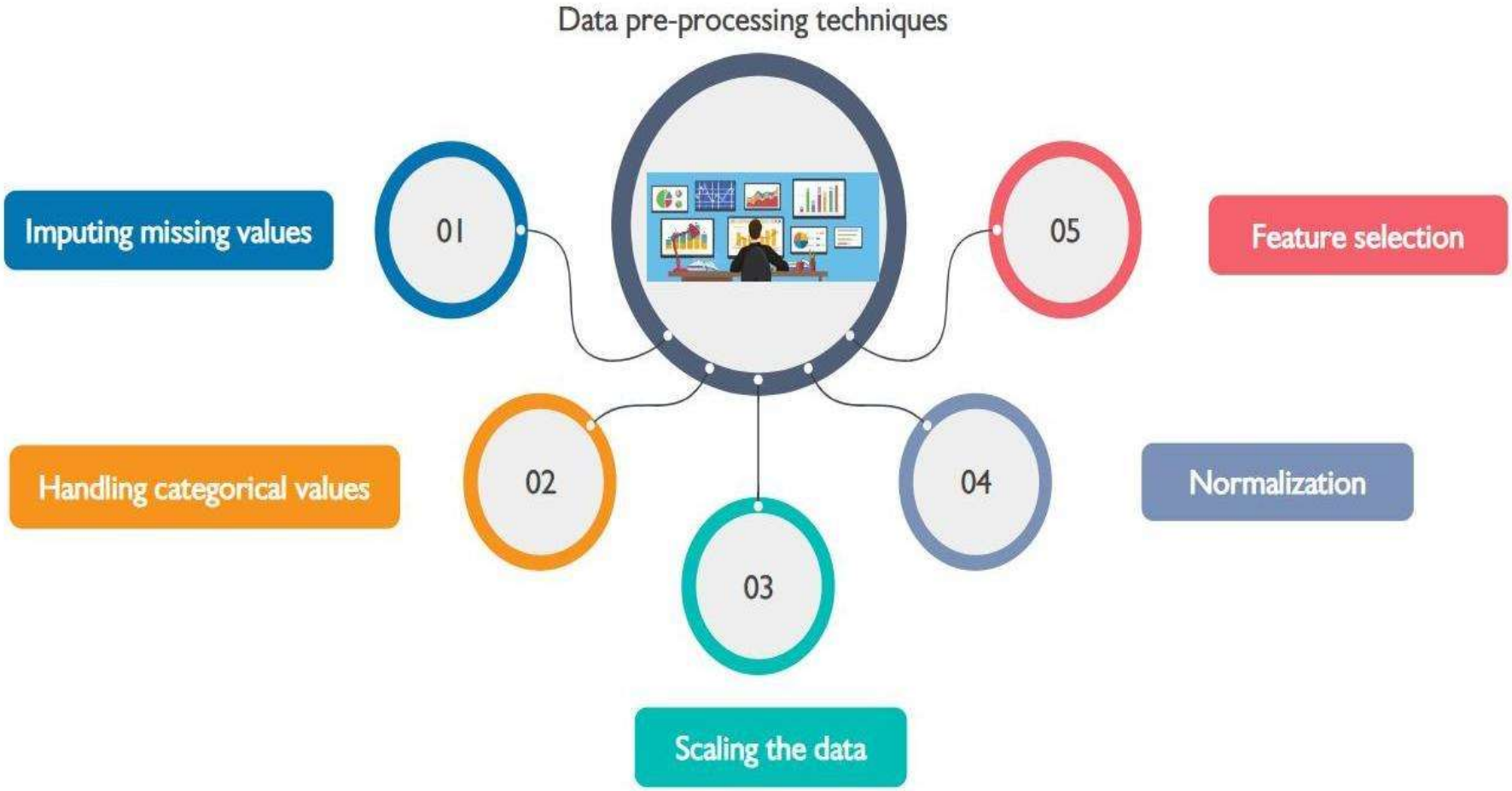
Data acquired from different sources



Clean data

Data filtering

Basics of Data Pre-processing Techniques



Imputing Missing Values: `SimpleImputer()`

- It is the process of replacing the missing values with the mean imputation.
- This can be done by importing `SimpleImputer()`.

Code example

```
import pandas
import numpy as np
data = {'one': pandas.Series([1, 2, 5],
                             index=['a', 'b', 'e']),
        'two': pandas.Series([1, 2, 3, 4],
                             index=['a', 'b', 'c', 'd'])}
table = pandas.DataFrame(data)
print("Before:\n",table)

from sklearn.impute import SimpleImputer
imputer = SimpleImputer(missing_values=np.nan, strategy='mean')
imputer.fit(table)
imputed_data = imputer.transform(table.values)
print("After:\n",pandas.DataFrame(imputed_data, columns=['one','two']))
```

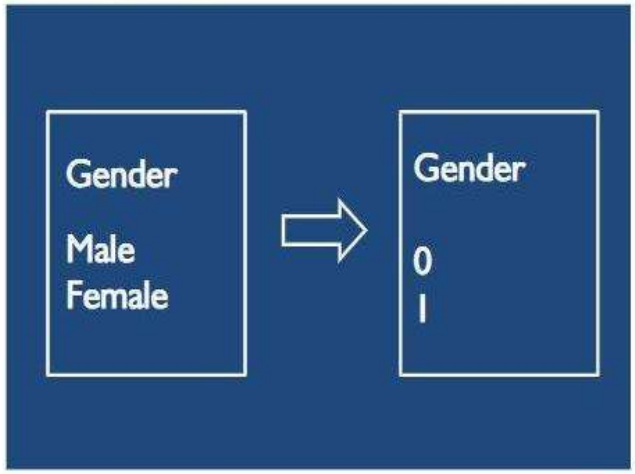
Code output

```
Before:
    one  two
a  1.0  1.0
b  2.0  2.0
c  NaN  3.0
d  NaN  4.0
e  5.0  NaN
After:
         one  two
0  1.000000  1.0
1  2.000000  2.0
2  2.666667  3.0
3  2.666667  4.0
4  5.000000  2.5
```

Handling the Categorical Values

- Categorical values are often represented as a column of String. But most of the machine learning libraries require the input to be in numerical format. Hence, we need to convert categorical values to numerical values.
- We can handle these categorical values using 3 methods:
  - `get_dummies()`
  - `LabelEncoder()`
  - `OneHotEncoder()`

Converting categorical values to numerical values



Handling the Categorical Values (contd.)

You shall handle the various categorical values on loan-info data-frame.

Code example

```
import pandas as pd
data = {'Customer_id': pd.Series([1,2,3,4,5]),
        'Loan_type': pd.Series(['Home Loan',
                                'Personal Loan',
                                'Education Loan',
                                'Home Loan',
                                'Credit Loan']),
        'Income': pd.Series(['30K','25K',
                             '15K','40K','35K'])}
loan_info = pd.DataFrame(data)
print(loan_info)
```

Code output

```
   Customer_id       Loan_type Income
0            1       Home Loan    30K
1            2   Personal Loan    25K
2            3  Education Loan    15K
3            4       Home Loan    40K
4            5     Credit Loan    35K
```

Handling the Categorical Values: `get_dummies()`

- `get_dummies()` gets the dummy variables for the categorical variables.
- It can be directly applied on the DataFrame.

Code example

```
import pandas as pd
data = {'Customer_id': pd.Series([1,2,3,4,5]),
        'Loan_type': pd.Series(['Home Loan',
                                'Personal Loan',
                                'Education Loan',
                                'Home Loan',
                                'Credit Loan']),
        'Income': pd.Series(['30K','25K',
                             '15K','40K','35K'])}
loan_info = pd.DataFrame(data)
loan_info = pd.get_dummies(loan_info,
                           prefix_sep='_',
                           drop_first=True)
print(loan_info)
```

Code output

```
   Customer_id  Loan_type_Credit Loan  Loan_type_Education Loan  \
0            1                      0                         0
1            2                      0                         0
2            3                      0                         1
3            4                      0                         0
4            5                      1                         0

   Loan_type_Home Loan  Loan_type_Personal Loan  Income_15K  Income_25K  \
0                    1                        0           0           0
1                    0                        1           0           1
2                    0                        0           1           0
3                    1                        0           0           0
4                    0                        0           0           0

   Income_30K  Income_35K  Income_40K
0           1           0           0
1           0           0           0
2           0           0           0
3           0           0           1
4           0           1           0
```

Handling the Categorical Values: `LabelEncoder()`

Label encoder is applied on categorical variables that have ordinal data when the number of categorical features are high.

Code example

```
import pandas as pd
data = {'Customer_id': pd.Series([1,2,3,4,5]),
        'Loan_type': pd.Series(['Home Loan',
                                'Personal Loan',
                                'Education Loan',
                                'Home Loan',
                                'Credit Loan']),
        'Income': pd.Series(['30K','25K','15K',
                             '40K','35K'])}
loan_info = pd.DataFrame(data)

from sklearn.preprocessing import LabelEncoder
labelencoder = LabelEncoder()
loan_info_upd=loan_info.apply(labelencoder.fit_transform)
print(loan_info_upd)
```

Code output

	Customer_id	Loan_type	Income
0	0	2	2
1	1	3	1
2	2	1	0
3	3	2	4
4	4	0	3

Handling the Categorical Values: `OneHotEncoder()`

- `OneHotEncoder()` is applied on categorical variables that do not have ordinal data.
- It is advantageous when the number of categorical features are less.

Code example

```
import pandas as pd
data = {'Customer_id': pd.Series([1,2,3,4,5]),
        'Loan_type': pd.Series(['Home Loan',
                                'Personal Loan',
                                'Education Loan',
                                'Home Loan',
                                'Credit Loan']),
        'Income': pd.Series(['30K','25K','15K',
                             '40K','35K'])}
loan_info = pd.DataFrame(data)

from sklearn.preprocessing import OneHotEncoder
onehotencoder = OneHotEncoder()
X = onehotencoder.fit_transform(loan_info).toarray()
print(X)
```

Code output

```
[[1. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 1. 0. 0.]
 [0. 1. 0. 0. 0. 0. 0. 0. 1. 0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0. 0. 1. 0. 0. 1. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0. 0. 1. 0. 0. 0. 0. 0. 1.]
 [0. 0. 0. 0. 1. 1. 0. 0. 0. 0. 0. 0. 1. 0.]]
```

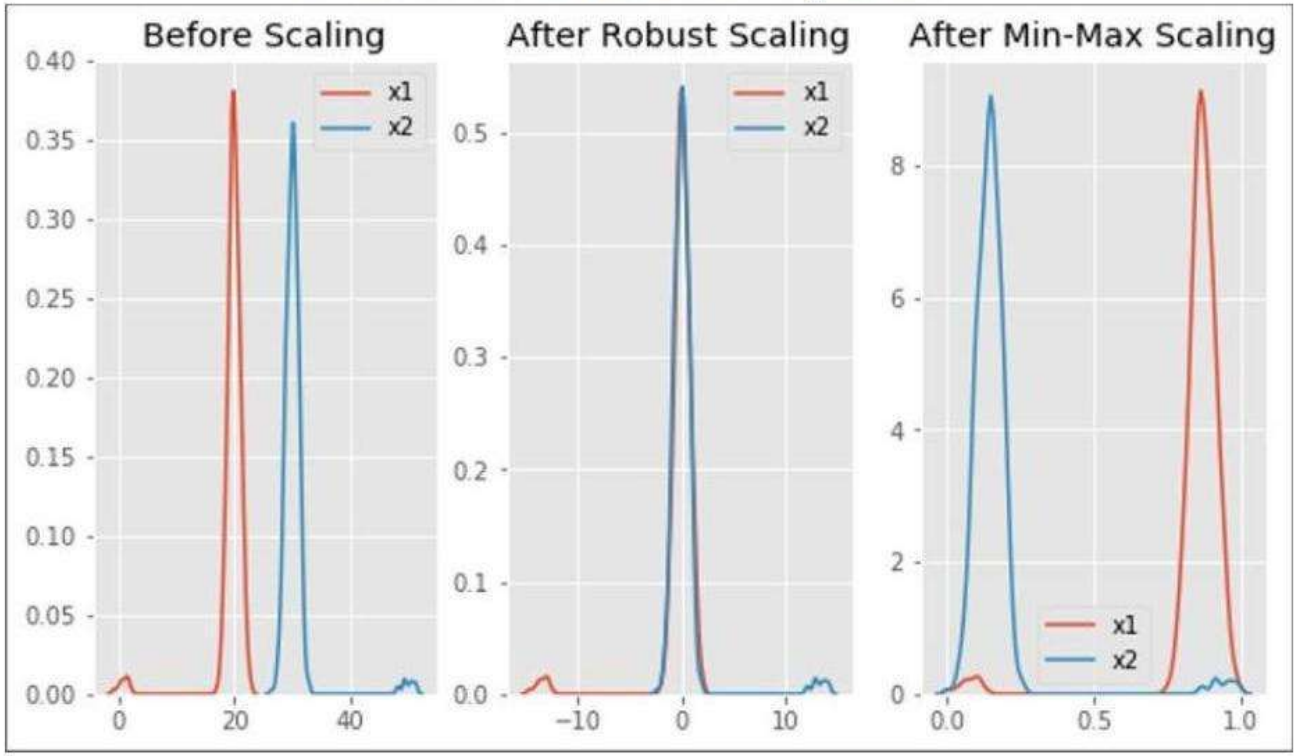
Data Pre-processing Techniques – II

Fundamentals of Data Pre-processing Techniques

Scaling the data

- Scaling is an approach that allows us to limit the range of variables so that they can be compared on common grounds.
- You can scale the data in 3 ways:
  - `StandardScaler()`
  - `MinMaxScaler()`
  - `RobustScaler()`

Distribution before and after scaling the data



Scaling: `StandardScaler()`

It helps in transforming a feature to have a mean of 0 and a standard deviation of 1.

Code example

```
import numpy as np
from sklearn import preprocessing

data1 = np.array([[-100.3],
                  [27.5],
                  [0],
                  [-200.9],
                  [1000]])
print('Before Scaling\n',data1)

standard_scaler = preprocessing.StandardScaler()
scaled = standard_scaler.fit_transform(data1)
print('\nAfter standard scaler\n',scaled)
```

Code output

```
Before Scaling
 [[-100.3]
 [  27.5]
 [   0. ]
 [-200.9]
 [1000. ]]

After standard scaler
 [[-0.5646401 ]
 [-0.27077707]
 [-0.33401051]
 [-0.79595951]
 [ 1.96538718]]
```

Scaling: `MinMaxScaler()`

It is the most famous scaling algorithm. It rescales the value of a numerical feature to be between two values.

Code example

```
import numpy as np
from sklearn import preprocessing

data1 = np.array([[-100.3],
                  [27.5],
                  [0],
                  [-200.9],
                  [1000]])
print('Before scaling\n',data1)

minmax_scale = preprocessing.MinMaxScaler(feature_range=(1, 2))
scaled = minmax_scale.fit_transform(data1)
print('\nAfter Min-Max Scaler\n',scaled)
```

Code output

```
Before scaling
 [[-100.3]
 [  27.5]
 [   0. ]
 [-200.9]
 [1000. ]]

After Min-Max Scaler
 [[1.08377051]
 [1.19019069]
 [1.1672912 ]
 [1.        ]
 [2.        ]]
```

Scaling: `RobustScaler()`

`RobustScaler` is similar to `MinMaxScaler()`, but it uses the interquartile range rather than the min–max, so that it is robust to outliers.

Code example

```
import numpy as np
from sklearn import preprocessing

data1 = np.array([[-100.3],
                  [27.5],
                  [0],
                  [-200.9],
                  [1000]])
print('Before Scaling\n',data1)

robust_scaler = preprocessing.RobustScaler()
scaled = robust_scaler.fit_transform(data1)
print('\n After Robust Scaler\n',scaled)
```

Code output

```
Before Scaling
 [[-100.3]
 [  27.5]
 [   0. ]
 [-200.9]
 [1000. ]]

 After Robust Scaler
 [[-0.78482003]
 [ 0.21517997]
 [ 0.        ]
 [-1.57198748]
 [ 7.82472613]]
```

Normalization

It is the process of scaling individual samples to have unit norm.

Code example

```
import numpy as np
from sklearn import preprocessing

data = np.array(
    [[5.1,3.5,1.4,0.2],
     [4.9,3.0,1.4,0.2],
     [4.7,3.2,1.3,0.2],
     [4.6,3.1,1.5,0.2],
     [5.0,3.6,1.4,0.2]])
print('Before Normalization\n',data)

normalized_data = preprocessing.normalize(data)
print('\nAfter Normalization\n',normalized_data)
```

Code output

```
Before Normalization
 [[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]

After Normalization
 [[0.80377277 0.55160877 0.22064351 0.0315205 ]
 [0.82813287 0.50702013 0.23660939 0.03380134]
 [0.80533308 0.54831188 0.2227517  0.03426949]
 [0.80003025 0.53915082 0.26087943 0.03478392]
 [0.790965   0.5694948  0.2214702  0.0316386 ]]
```

Normalization divides the data into a common scale without much difference between them.

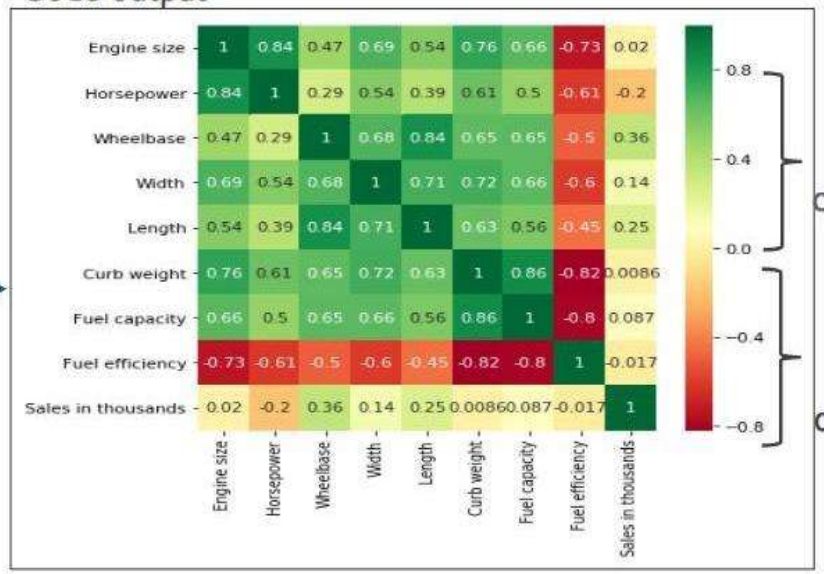
Feature Selection

- It is a technique of selecting the relevant variables to use in model selection.
- Negative correlation impacts the performance of the model.

Code example

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
#import the dataset
df = pd.read_csv("CarSales.csv")
#drop the unwanted columns
df=df.drop(['4-year resale value','Price in thousands'],axis=1)
#get correlations of each features in dataset
corr_mat = df.corr()
top_corr_features = corr_mat.index
#plot the figure using matplot lib
plt.figure(figsize=(6,6))
#plot heat map using seaborn
x=sns.heatmap(df[top_corr_features].corr(),annot=True,cmap="RdYlGn")
```

Code output



High correlation

Low correlation

Testing and Training Data

Splitting the Data: Train/Test Split

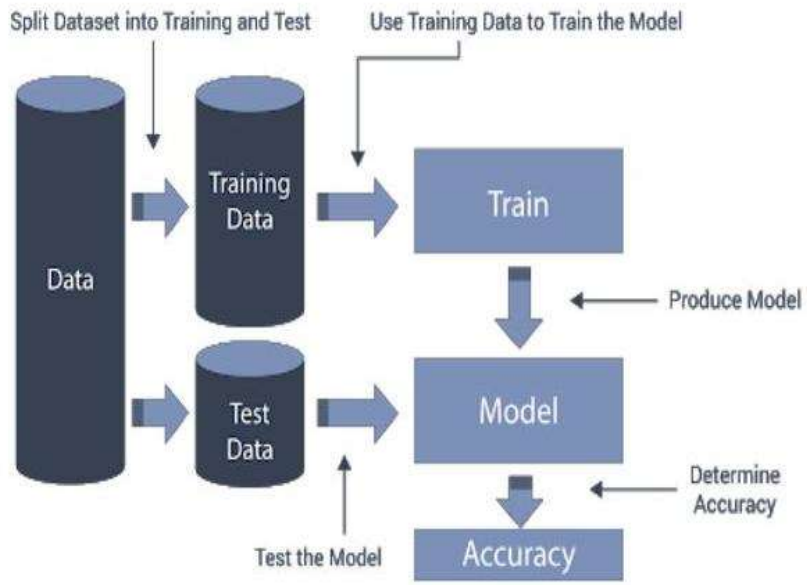
Machine learning algorithms divide the data-sets into a training set and a testing set.

Test-set conditions

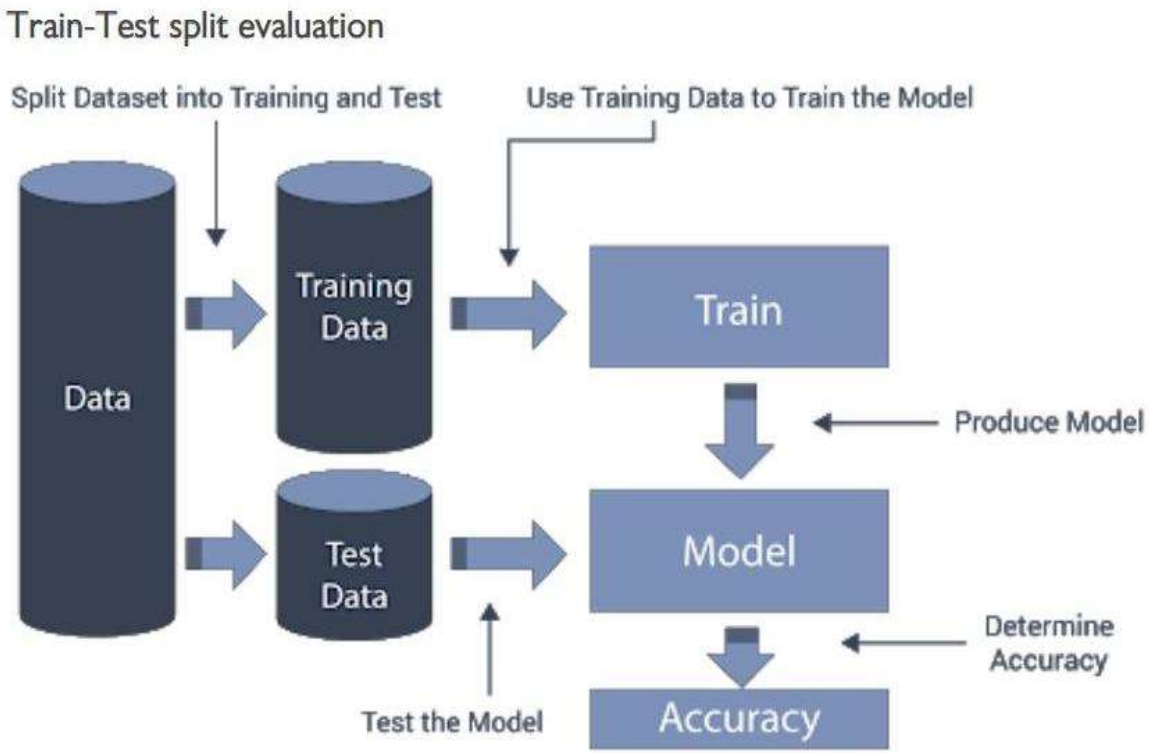
- Should be able to provide meaningful results statistically
- Should represent the data-set as a whole i.e., with the same characteristics as a training set

Note: Usually the data is split in an 80% (training set) – 20% (testing set) fashion and then the model is trained.

Working of the algorithm with data



Splitting the Data: Train/Test Split (contd.)



Splitting the Data: Train/Test Split (contd.)

Importing the appropriate libraries and reading the data set for splitting the data.

Code example

```
#importing libraries
import pandas as pd
from sklearn.model_selection import train_test_split
#read the data-sets
df=pd.read_csv('cereal.csv')
df.head()
```

Code output

	name	mfr	type	calories	protein	fat	sodium	fiber	carbo	sugars	potass	vitamins	shelf	weight	cups	rating
0	100% Bran	N	C	70	4	1	130	10.0	5.0	6	280.0	25	3	1.0	0.33	68.402973
1	100% Natural Bran	Q	C	120	3	5	15	2.0	8.0	8	135.0	0	3	1.0	1.00	33.983679
2	All-Bran	K	C	70	4	1	260	9.0	7.0	5	320.0	25	3	1.0	0.33	59.425505
3	All-Bran with Extra Fiber	K	C	50	4	0	140	14.0	8.0	0	330.0	25	3	1.0	0.50	93.704912
4	Almond Delight	R	C	110	2	2	200	1.0	14.0	8	NaN	25	3	1.0	0.75	34.384843

Splitting the Data: Train/Test Split (contd.)

The dataset is restructured and split in 80–20 fashion.

Code example

```
#Restructuring the Dataframe
x=df[['calories','protein']].values
y=df['rating'].values
# Splitting the data-set into training and testing into 80%-20%
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2)
# train data-set is observed to be 80% of data
print('Trained data-set:',x_train.shape)
#test data-set is observed to be 20% of data
print('Test data-set:',x_test.shape)
```

Code output

```
Trained data-set: (61, 2)
Test data-set: (16, 2)
```